TP 16 : Tas binomiaux

Ce TP est à réaliser dans son entièreté en C.

Préliminaires

Un arbre binomial est défini récursivement de la façon suivante :

- l'arbre binomial d'ordre 0 est un simple nœud.
- l'arbre binomial d'ordre k possède une racine de degré k et ses fils sont des arbres binomiaux d'ordre $k-1$, $k-2$, …, 2, 1, 0. Ceux-ci sont ordonnés par ordre croissant.

Un arbre binomial d'ordre k peut aussi être construit à partir de deux arbres d'ordre $k-1$ en faisant de l'un des deux le fils le plus à gauche de la racine de l'autre. Un arbre binomial d'ordre k possède 2^k nœuds, et a pour hauteur k. Vous pouvez essayer de le démontrer à partir de la définition si vous le souhaitez.

Un tas binomial est un ensemble d'arbres binomiaux satisfaisant certaines propriétés. Ce n'est donc absolument pas un arbre binaire presque complet comme ce qu'on a pu voir en cours et l'on fera attention à ne pas mélanger les deux notions.

On propose d'utiliser un type récursif simple pour implémenter les arbres composant un tas binomial. Chaque nœud est étiqueté à l'aide de 3 entiers : sa clef, sa valeur et l'ordre de l'arbre binomial enraciné en celui-ci.

```
typedef struct arbre =
{
    int clef;
    int valeur;
    int ordre;
    struct arbre* liste_enfants;
    struct arbre* suivant;
}arbre_binomial;

typedef arbre_binomial* tas_binomial;
```

Chaque objet de type `arbre_binomial` est en réalité un maillon d'une liste chaînée d'arbres binomiaux. L'attribut `suivant` est un pointeur vers le maillon suivant, celui de l'arbre suivant la fratrie tandis que l'attribut `liste_fils` est une autre liste chaînée d'arbres binomiaux, les enfants du nœud considéré. La structure de tas binomial correspond simplement à une liste chaînée d'arbres, l'attribut `suivant` permettant de faire le lien entre les différents éléments de la liste.

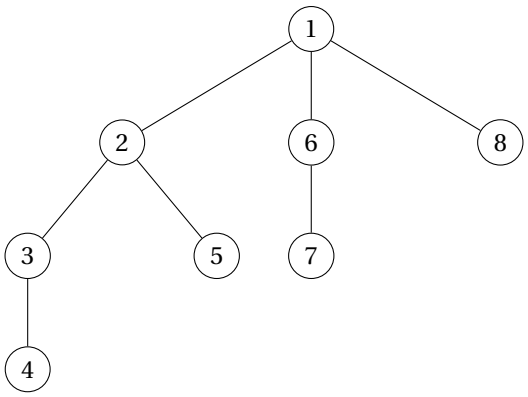

FIGURE 1 – Arbre binomial d'ordre 3

Sur l'exemple de la figure 1, on a représenté un arbre binomial d'ordre 3. L'attribut `suivant` du nœud de clef 2 pointe vers celui de clef 6 tandis que sont attribut `liste_enfants` pointe vers le nœud de clef 3.

Question 1 : Écrire une fonction `arbre_binomial* creer_noeud(int clef, int valeur, int ordre)` qui alloue l'espace mémoire nécessaire pour un nœud d'arbre binomial et initialise les attributs fournis. Les autres seront initialisés à `NULL`.

Correction :

```
arbre_binomial* creer_noeud(int clef, int valeur, int ordre)
{
    arbre_binomial* res = malloc(sizeof(arbre_binomial));
    res->clef = clef;
    res->valeur = valeur;
    res->ordre = ordre;
    res->liste_enfants = NULL;
    res->suivant = NULL;
    return res;
}
```

Question 2 : Écrire une fonction `arbre_binomial* exemple(int n)` qui, étant donné un entier n, renvoie un arbre binomial d'ordre n dont toutes les clefs et valeurs valent 1.

Correction :
Il y a deux principales façon de faire :

- Soit on utilise la remarque du sujet : On crée deux arbres binomiaux d'ordre n et l'on ajoute le premier aux fils du second.

```
arbre_binomial* exemple(int n)
{
    if (n==0)
    {
        return creer_noeud(1, 1, n);
    }
    arbre_binomial* a1 = exemple(n-1);
    arbre_binomial* a2 = exemple(n-1);
    a2->suivant = a1->liste_fils;
    a1->liste_fils = a2;
    return a1;
}
        \item Soit on crée les $n$ arbres binomiaux fils dans une
            liste. On conserve dans une variable le dernier sous-
            arbre pour lui ajouter son prochain frère.
        \begin{tcolorbox}[colback=white]
            \begin{lstlisting}
arbre_binomial* exemple(int n)
{
    arbre_binomial* res = creer_noeud(1, 1, n);
    if (n == 0)
    {
        return res;
    }
    arbre_binomial* dernier_enfant = exemple(n-1);
    res->liste_enfants = dernier_enfant;
    for (int i = n-2; i > -1; i--)
    {
        dernier_enfant->suivant = exemple(i);
        dernier_enfant = dernier_enfant->suivant;
    }
    return res;
}
```

Tas binomiaux

Les arbres composant un tas binomial vérifient les propriétés supplémentaires suivantes :

- La propriété de tas-minimum, c'est-à-dire que la clef d'un nœud est toujours inférieure ou égale à celle de ses fils. La clef minimum d'un tas binomial se trouve donc à la racine de l'un de ses arbres.
- Pour chaque entier j, il existe au plus un arbre binomial d'ordre j au sein du tas binomial.
- Les arbres binomiaux sont classés selon leur ordre croissant.

Néanmoins, les opérations que l'on souhaite réaliser sur cette structure de données sont similaires :

- **Insertion**
- **Extraire le minimum**
- **Décroître une clef**
- **Unir deux tas**

Question 3 : Écrire une fonction `void liberer_tas(tas_binomial t)` qui, étant donné un tas binomial, libère l'espace mémoire alloué pour celui-ci. On veillera à bien libérer l'espace mémoire utilisé par chacun des arbres.

Correction :
Pour un tas non vide, on libère les sous-arbres du premier arbre et les autres arbres du tas à l'aide d'appel récursifs. Enfin, on libère l'espace mémoire réservé par le nœud de départ.

```
void liberer(tas_binomial t)
{
    if (t!=NULL)
    {
        liberer(t->liste_enfants);
        liberer(t->suivant);
        free(t);
    }
}
```

Réaliser l'union

Comme son nom ne l'indique pas du tout, l'union est l'opération la plus complexe à implémenter sur les tas binomiaux. En effet, on verra que les autres y font appel. L'union ressemble en quelque sorte à la fusion de deux listes triées dans le tri fusion. On parcours les deux listes en parallèle dans l'ordre croissant et l'on construit un arbre d'ordre $j+1$ à chaque fois que l'on réalise que l'on a deux arbres d'ordre j.

Question 4 : Écrire une fonction `arbre_binomial* fusion(arbre_binomial* a1, arbre_binomial* a2)` qui, étant donnés deux arbres binomiaux de même ordre j respectant la propriété de tas-minimum, renvoie un arbre binomial d'ordre $j+1$ respectant la propriété de tas-minimum qui contient exactement les mêmes éléments que l'union des éléments des deux arbres initiaux. On pourra utiliser l'une des définitions possibles des arbres binomiaux.

Correction :
On utilise la remarque du début du sujet pour, à partir de deux arbres binomiaux d'ordre n, obtenir un arbre binomiaux d'ordre $n+1$ contenant les mêmes éléments. On teste les clefs des racines pour s'assurer que la propriété de tas-minimum est préservée.

```
arbre_binomial* fusion(arbre_binomial* a1, arbre_binomial* a2)
{
    if (a1->clef < a2->clef)
    {
        a2->suivant = a1->liste_enfants;
        a1->liste_enfants = a2;
        a1->ordre++;
        return a1;
    }
    else
    {
        a1->suivant = a2->liste_enfants;
        a2->liste_enfants = a1;
        a2->ordre++;
        return a2;
    }
}
```

Question 5 : Écrire une fonction `tas_binomial fusionner_doublons(tas_binomial t)` qui, étant donné une instance de `tas_binomial` où la seconde condition n'est pas vérifiée, réalise un parcours de la liste en fusionnant les arbres dont les ordres sont identiques. On veillera à conserver les autres conditions.

Correction :

```
tas_binomial fusionner_doublons(tas_binomial t)
{
    if (t == NULL || t->suivant == NULL)
    {
        return t;
    }
    if (t->ordre != t->suivant->ordre)
    {
        t->suivant = fusionner_doublons(t->suivant);
        return t;
    }
    arbre_binomial* a1 = t;
    arbre_binomial* a2 = t->suivant;
    tas_binomial suite = a2->suivant;
    a2->suivant = NULL;
    a1->suivant = NULL;
    arbre_binomial* a = fusion(a1, a2);
    a->suivant = suite;
    return fusionner_doublons(a);
}
```

Question 6 : Écrire une fonction `tas_binomial unir(tas_binomial t1, tas_binomial t2)` qui réalise l'union des tas binomiaux passés en paramètre.

Correction :
Comme expliqué en TD, l'objectif ici est de concaténer les deux listes en veillant à ne pas créer plusieurs arbres de même ordre. Il y a ici encore, deux façons de gérer les collisions créées par les arbres obtenus par fusion.

- Soit ils sont ajoutés astucieusement à l'aide d'un appel récursif à `unir` sur un tas avec uniquement l'arbre fraîchement créé.

```
tas_binomial unir(tas_binomial t1, tas_binomial t2)
{
    if (t1 == NULL)
    {
        return t2;
    }
    if (t2 == NULL)
    {
        return t1;
    }
    if (t1->ordre < t2->ordre)
    {
        t1->suivant = unir(t1->suivant, t2);
        return t1;
    }
    else if (t2->ordre < t1->ordre)
    {
        t2->suivant = unir(t1, t2->suivant);
        return t2;
    }
    else
    {
        tas_binomial suite1 = t1->suivant;
        tas_binomial suite2 = t2->suivant;
        t1->suivant = NULL;
        t2->suivant = NULL;
        arbre_binomial* fusionne = fusion(t1, t2);
        return unir(fusionne, unir(suite1, suite2));
    }
}
```

- Soit ils sont ajoutés naïvement puis un deuxième passage fusionne alors les doublons. Il faut cependant s'assurer qu'un seul passage suffit. Cela est bien le cas puisqu'on peut montrer qu'il n'y aura pas deux doublons d'ordres successifs comme : $\{A_4, A_4, A_5, A_5\}$.

```
tas_binomial unir_aux(tas_binomial t1, tas_binomial t2)
{
    if (t1 == NULL)
    {
        return t2;
    }
    if (t2 == NULL)
    {
        return t1;
    }
    if (t1->ordre < t2->ordre)
    {
        t1->suivant = unir_aux(t1->suivant, t2);
        return t1;
    }
    else
    {
        t2->suivant = unir_aux(t1, t2->suivant);
        return t2;
    }
}

tas_binomial unir(tas_binomial t1, tas_binomial t2)
{
    return fusionner_doublons(unir_aux(t1,t2));
}
```

Autres opérations

Question 7 : Écrire une fonction `tas_binomial insertion(tas_binomial t1, int clef, int valeur)` qui, étant donné un tas binomial, une clef et la valeur associée, insère celles-ci dans le tas. On pourra chercher à utiliser la fonction précédente sur des tas binomiaux bien choisis.

Correction :
Il suffit de construire un tas binomial avec comme unique élément celui que l'on cherche à insérer. On peut alors utiliser la fonction `unir` réalisée plus tôt pour l'insérer.

```
tas_binomial insertion(tas_binomial t1, int clef, int valeur)
{
    arbre_binomial* a = creer_noeud(clef, valeur, 0);
    return unir(t1, a);
}
```

Question 8 : Écrire une fonction `int trouver_min(tas_binomial t1)` qui étant donné un tas binomial, renvoie la valeur dont la clef est minimale. On utilisera la propriété de tas-minimum que possèdent tous les arbres binomiaux composant le tas.

Correction :
On sait que le minimum d'un arbre à l'intérieur d'un tas binomial est forcément situé à la racine. Il suffit donc de comparer toutes les racines pour trouver le plus petit élément du tas.

```
int trouver_min(tas_binomial t)
{
    int mini_val = t->valeur;
    int mini_clef = t->clef;
    while (t!=NULL)
    {
        if (t->clef<mini_clef)
        {
            mini_clef=t->clef;
            mini_val=t->valeur;
        }
        t=t->suivant;
    }
    return mini_val;
}
```

Question 9 : Écrire une fonction `tas_binomial extraire_min(tas_binomial t)` qui étant donné un tas binomial, renvoie le tas binomial dont le couple de clef minimale a été retiré.

Correction :
Pour retirer le plus petit élément, on récupère tous les fils de la racine que l'on unit avec le reste du tas. Il faut juste prendre soin d'avoir les fils par ordre croissant.

```c
tas_binomial retourner(tas_binomial t)
{
    if (t->suivant == NULL)
    {
        return t;
    }
    tas_binomial precedent = t;
    t = t->suivant;
    precedent->suivant = NULL;
    while(t != NULL)
    {
        tas_binomial temp = t->suivant;
        t->suivant = precedent;
        precedent = t;
        t = temp;
    }
    return precedent;
}

tas_binomial extraire_min(tas_binomial t)
{
    int mini_val = trouver_min(t);
    if (t->valeur == mini_val)
    {
        return unir(t->suivant, retourner(t->liste_enfants));
    }
    tas_binomial temp = t->suivant;
    tas_binomial precedent = t;
    while (temp->valeur != mini_val)
    {
        temp=temp->suivant;
    }
    precedent->suivant = temp->suivant;
    temp->suivant = NULL;
    tas_binomial element_retire = temp->liste_enfants;
    free(temp);
    return unir(retourner(element_retire), t);
}
```

Question 10 : Expliquer pourquoi il est compliqué d'implémenter l'opération de diminution de la clef avec un tel choix d'implémentation de structure de données.

Correction :
Pour trouver la clef associée à une valeur, il faudrait savoir où celle-ci se trouve dans le tas. Or, il n'y a pas de façon d'être sûr de trouver une clef sans avoir à explorer tous les arbres du tas. De plus, l'opération de diminution de la clef d'un nœud peut faire remonter celui-ci. Il faudra alors ajouter un attribut `parent` pour pouvoir faire cela.

Question 11 : Quitte à modifier certaines choses, proposer un nouveau type permettant d'implémenter une telle méthode.

Correction :
On pourra utiliser une méthode proche de celle du tas classique. On ajouterai un tableau qui permettrait, étant donné une valeur, de savoir où celle-ci se trouve dans le tas. Voici une proposition de type en `C`. Ici, j'ai considéré que les valeurs étaient entières et bornées (comme les sommets d'un graphe). On utiliserai dans le cas général une table de hachage ou une autre représentation de dictionnaire.

```
typedef struct arbre
{
    int clef;
    int valeur;
    int ordre;
    struct arbre* liste_enfants;
    struct arbre* suivant;
    struct arbre* parent;
}arbre_binomial;

struct tas_binomial
{
    struct arbre_binomial* liste_arbres;
    struct arbre_binomial** tableau_correspondance;
};
```

Il suffirait alors d'utiliser le tableau `tableau_correspondance` pour savoir quel nœud du tas modifier. Après diminution de la valeur de la clef, on doit alors percoler vers la racine en maintenant le tableau des correspondances. Cela est réalisé en $O(\log n)$ car le plus grand des arbres binomiaux a une hauteur d'au plus $O(\log n)$.

Question 12 : Expliquer comment il est alors possible de retirer un élément arbitraire dans un tas binomial.

Correction :
Il suffirait de récupérer la clef minimale. On peut alors suffisamment diminuer la clef de l'élément que l'on souhaite retirer pour qu'il soit le plus petit. On peut alors l'extraire à l'aide de la fonction `extraire_minimum`.

Pour aller plus loin

Les tas de Fibonacci sont une amélioration des tas binomiaux qui permettent d'avoir une union de deux tas plus efficace. Si cela vous intéresse, vous pouvez regarder cette vidéo : `https://www.youtube.com/watch?v=6JxvKfSV9Ns`. C'est en anglais et ça fait réviser (un peu) la complexité amortie.